Range-Filtering Approximate Nearest Neighbor Search via First-Split Range Decomposition and Suffix-Prefix Indexing

Yufan Xiang[1,*], Junyao Wang[2,*], Chen Feng[2,#], and Jieming Shi[2]
[1]University of Wisconsin–Madison, [2]The Hong Kong Polytechnic University
Email: yxiang45@wisc.edu; junya.wang@connect.polyu.hk; chenfeng@polyu.edu.hk; jieming.shi@polyu.edu.hk

***Abstract*—Given a query vector and n input data objects, range-filtering approximate nearest neighbor search (RFANNS) aims to find its nearest neighbors whose numeric attributes fall within a query range, supporting various applications in databases, information retrieval, and machine learning. Existing methods for RFANNS either consult more than a constant number of indexes per query or build an index over a range evidently larger than the query range. The two drawbacks impair their scalability to large-scale queries.**

To improve search efficiency, we propose `FRASP`, a novel method that employs a first-split range strategy with suffix and prefix HNSWs in a segment tree. `FRASP` answers any query with only three segment-specific graph indexes and evaluates edges per object with running time linear with the maximum degree in graph indexes. The superior performance of `FRASP` is achieved by four key designs. (i) We apply the first-split position in the segment tree to split the query range into a suffix and a prefix range, contained by the left and right child segments of a parent segment respectively. (ii) During index construction, we build suffix HNSWs for left child segments and prefix HNSWs for right child segments, enabling arbitrary query ranges to be supported by two one-sided ranges. (iii) A unified search is performed over the suffix, prefix, and parent segments, to explore objects within the query range without redundant computations. (iv) We design prefix/suffix max arrays to locate relevant HNSW layers and initial entry points efficiently via simple array lookup. Extensive experiments indicate that FRASP outperforms state-of-the-art methods in terms of both query efficiency and result quality.

***Index Terms*—approximate nearest neighbor search, range filtering, graph index, HNSW, vector database**

I. Introduction

In the era of machine learning and big data, many real-world entities are represented as vectors to support downstream tasks. Efficient management of high-dimensional vector data becomes an essential technique in many applications, including recommendation systems [1], information retrieval [2], machine learning systems [3], [4], language models [5], and database systems [6]. A fundamental task in vector processing is nearest neighbor search (NNS), which aims to find the closest vectors to a query from a large dataset based on a distance metric such as Euclidean distance.

With the rapid growth of vector dimension and dataset size, exact nearest neighbor search (NNS) becomes computationally expensive and incurs high latency. To improve efficiency, the distance requirement is relaxed to approximate NNS (ANNS), which finds vectors that are approximately nearest to the query. This trade-off significantly improves query efficiency while maintaining high result quality. Various indexes have been developed to support ANNS, many of which are based on Hierarchical Navigable Small World (HNSW) graphs [7], achieving high recall and throughput [8]–[10].

In industrial scenarios and vector databases [11], [12], data objects often include numerical attributes (e.g., timestamps or prices) alongside vectors. Many applications require joint querying of vectors and their attributes. For example, on e-commerce platforms, a user can upload a query product image and specify a price range to find similar products within the range. On video platforms, video recommendations based on a query vector (derived from a user's watch history) must comply with age-related regulations by applying an age filter. Related examples also include image-similarity search filtered by image width (e.g., LAION), or similar-word search over word embeddings with a word-frequency range (e.g., GloVe-Twitter) [13]–[15]. These queries are known as range-filtering ANNS (RFANNS), aiming to find the nearest neighbors of a query vector whose attributes fall within the specified range filter (i.e., *in-range*), while excluding *out-of-range* objects.

A profusion of methods are proposed to solve RFANNS. Simple adaptations to previous methods (e.g., pre-filtering, post-filtering, and Milvus [11]) for ANNS are often inefficient, since they are not especially designed for RFANNS. Recent advancements [9], [10], [16], [17] have developed efficient indexes and query techniques dedicated for RFANNS. Their effectiveness depends on filter selectivity. For SeRF [10] and WoW [17], their indexes contain many out-of-range or invalid edges, which occupy extra space and should be skipped during search. UNIFY [18] and iRangeGraph [9] maintain many fragmented indexes. To elaborate, UNIFY may examine $O(S)$ segment-wise chunks in hybrid filtering, with $S = 8$ in its default configuration, and iRangeGraph consults $O(\log n)$ graph indexes for each query. Thus, more efficient approaches should be designed to accelerate RFANNS.

In this paper, we propose an efficient method for RFANNS called `FRASP`[1], based on a segment tree which takes each

*Equal contribution. #Chen Feng is the corresponding author.

[1]First-split RAnge exploration using Suffix and Prefix HNSWs

segment as a node and each segment is recursively split at its midpoint. To achieve high efficiency, FRASP involves four techniques tailored for RFANNS. (1) **The first-split strategy**. Given a query range $[L, R]$, we first find from the segment tree the largest segment $[l, r]$ whose midpoint (denoted as fs) is in $[L, R]$. Then, the query range is split into two ranges $[L, fs]$ and $(fs, R]$, called *suffix range* and *prefix range* resp, which are contained by the children of $[l, r]$ (i.e., $[l, fs]$ and $(fs, r]$ resp.). In this way, we only need to focus on three segments $[l, r]$, $[l, fs]$, and $(fs, r]$ for a query. (2) **Suffix/prefix HNSW**. For efficient search in any segment, we build an HNSW for each segment, whose role in the segment tree is a left child or a right child. If a segment is a left child, we construct a suffix HNSW for it. Otherwise, we build a prefix HNSW for the segment. By default, the root segment maintains a prefix HNSW. (3) **Unified search**. To leverage the inter-edges between child segments, we utilize the HNSWs of not only the child segments but also the parent segment $[l, r]$. On this basis, we design a unified search process over the parent segment and the two child segments, to share the intermediate results and a global lower bound for pruning and early termination. This unified search ensures exploration is restricted to in-range objects, effectively simulating a single HNSW search for the query range $[L, R]$, avoiding redundant computations. (4) **Prefix/Suffix Max Arrays**. Note that, HNSW is not originally designed for RFANNS. Directly searching from the top-most layer leads to unnecessary out-of-range computation. Thus, we design max array indexes that enable efficient identification of the entry points through simple array lookup. With the above four techniques, FRASP processes each query using only three segments and their HNSWs. Each processed object appears in at most two of these HNSWs, so at most $2M_0$ ordinary outgoing edges are accessed per object, where $M_0 = 2M$ is the maximum base-layer degree of HNSW. Thus, the expected query processing time per object is $O(M)$.

Extensive experiments on various benchmark datasets demonstrate that FRASP achieves significant improvements over existing methods, in terms of both efficiency and quality. In summary, our contributions are presented as follows.

- We propose the FRASP method to solve RFANNS efficiently. For each query, FRASP only consults three segments and their HNSWs, much less than existing methods.
- We design a first-split strategy to process query ranges in the segment tree. This strategy is not tied to HNSW and can be instantiated with ANNS indexes that support attribute-ordered insertion.
- Accordingly, we design suffix and prefix HNSWs for the query ranges. Moreover, we design prefix/suffix max arrays to efficiently identify the entry points of HNSWs.
- We design a unified search process over the three HNSWs, which maintains the connectivity between child segments and ensures efficient exploration within the query range.
- Experiments validate that FRASP outperforms state-of-the-art methods in terms of query efficiency and result quality.

TABLE I
FREQUENTLY USED NOTATIONS

Notation	Description
n	Number of objects in the dataset
$q, [L, R]$	Query vector in $\mathbb{R}^d$ and Query range
fs	First-split position in the segment tree for a query range
$\mathcal{D}$	Set of n objects in $\mathbb{R}^d$
$o.a$	Attribute value of object o
$[l, r]$	Segment in the segment tree
$\mathcal{H}_{\text{sfx}}$	Suffix HNSW for a left child segment
$\mathcal{H}_{\text{pfx}}$	Prefix HNSW for a right child segment
$\mathcal{H}_{\text{p}}$	HNSW for the parent segment
$N_b[o_i]$	Outgoing neighbors of o_i at the b-th layer
$N_b^{\text{del}}[o_i]$	Pruned (deleted) neighbors of o_i at the b-th layer
M	Maximum degree parameter in HNSW
ef, ef_{con}	Candidate list size for search/construction
A	Array of highest HNSW layers per object
Z	Suffix-Max (or Prefix-Max) array depending on the segment
$\mathcal{L}_a$	The a-th layer in HNSW
$\mathcal{L}_0$	Base layer in HNSW
$\mathcal{L}_{\text{qmax}}$	Topmost query-relevant HNSW layer or layer id
ep_{qmax}	Query-relevant entry point in $\mathcal{L}_{\text{qmax}}$
$\mathcal{C}$	Candidate priority queue in search
$\mathcal{R}$	Result priority queue in search
$\delta(\cdot, \cdot)$	Distance function (e.g., Euclidean)

II. PRELIMINARIES

A. Problem Definition

Given a set of n vectors $\mathcal{D} = \{o_1, o_2, \cdots, o_n\}$ and a query vector q in $\mathbb{R}^d$, the nearest neighbor search (NNS) problem aims to find the top k nearest neighbors in $\mathcal{D}$ under a certain distance metric δ, e.g., the Euclidean distance [19]. In many applications, each object $o \in \mathcal{D}$ is associated with a numeric attribute $o.a$ in addition to the vector data, such as size, time, or price. Accordingly, the aim becomes finding the nearest neighbors within a specific range $[a_L, a_R]$ of attribute values, formulated as the range-filtering NNS (RFNNS) problem.

Definition 1 (RFNNS [9], [10])**.** *Given a set $\mathcal{D}$ of objects in $\mathbb{R}^d$ where each object o is associated with an attribute $o.a$, a query vector $q \in \mathbb{R}^d$, and a query range $[a_L, a_R]$, the RFNNS query is to find a set Φ^* of k objects from $\mathcal{D}$ such that:*

1) $\forall o_i \in \Phi^* : o_i.a \in [a_L, a_R]$*, and*
2) $\forall o_i \in \Phi^*, \forall o_j \in \mathcal{D} \setminus \Phi^* : o_j.a \in [a_L, a_R] \Rightarrow \delta(q, o_i) \leq \delta(q, o_j)$.

Exact results are computationally expensive to compute, and thus *approximate RFNNS* (RFANNS) algorithms are often studied, aiming to efficiently find an approximate top-k result Φ that closely approximates the true nearest neighbors Φ^* within the query range. The performance is often measured by recall $|\Phi \cap \Phi^*|/|\Phi^*|$.

An object is referred to as *in-range* if its attribute is within the query range $[a_L, a_R]$, and as *out-of-range* otherwise. Following [9], [10], we assume that the objects in $\mathcal{D} = \{o_1, o_2, \ldots, o_n\}$ are sorted in ascending order of their attribute values. If not, the objects can be pre-sorted based on their attribute values. This sorting establishes a one-to-one mapping between the index of an object and its attribute value. That is, for any $o_i, o_j \in \mathcal{D}$, if $i < j$, then $o_i.a \leq o_j.a$. This allows the index of an object to represent its attribute value. Given a query range $[a_L, a_R]$, it is easy to determine

Algorithm 1: `InsertObject`(o)

Input: Entry point $ep_{\max}$ in $\mathcal{L}_{\max}$, ef_{con}, maximum connections M, the object o to be inserted
Output: Updated HNSW $\mathcal{H}$

1 $\mathcal{L}_a \leftarrow$ `SampleLevelRandomly`(M) `// Sample a level for` o
2 $ep \leftarrow ep_{\max}$;
3 **for** $\mathcal{L}_b$ *from* $\mathcal{L}_{\max}$ *to* $\mathcal{L}_{a+1}$ **do**
4 $\quad ep \leftarrow$ `SearchLayer`($o, ep, ef = 1, \mathcal{L}_b$);
5 **for** $\mathcal{L}_b$ *from* $\mathcal{L}_a$ *to* $\mathcal{L}_0$ **do**
6 $\quad \mathcal{W} \leftarrow$ `SearchLayer`($o, ep, ef_{\text{con}}, \mathcal{L}_b$);
7 $\quad N_b[o] \leftarrow$ Select at most M nearest neighbors from $\mathcal{W}$;
8 $\quad$ **foreach** $v \in N_b[o]$ **do**
9 $\quad\quad N_b[v] \leftarrow$ `PruneExistingEdges` ($N_b[v] \cup \{o\}$);
10 **return** Updated $\mathcal{H}$;

the smallest index L in $\mathcal{D}$ such that $o_L.a \geq a_L$, and the largest index R such that $o_R.a \leq a_R$. Thus, the problem is reduced to finding the k nearest neighbors of q among the objects in the index range $[L, R]$ of $\mathcal{D}$.

Table I summarizes the frequently used notations in this paper. Due to space constraints, the proofs of lemmas and theorems are deferred to our technical report [20].

B. The HNSW Index

Graph-based indexes (such as NSW [21], NSG [22], and Vamana [23]) exhibit superior performance for ANNS [7], [8], [24]. Their main idea is to build a graph called *proximity graph* where each vertex represents a vector and edges are established based on the distance between vectors. Based on NSW, HNSW organizes multiple layers in a hierarchical structure, where each layer is an NSW graph in essence. Given an HNSW $\mathcal{H}$, the layers are denoted as $\mathcal{L}_0, \ldots, \mathcal{L}_{\max}$, where $\mathcal{L}_0$ is the base layer and $\mathcal{L}_{\max}$ is the topmost layer. The base layer $\mathcal{L}_0$ contains all vectors, while each upper layer contains an exponentially decreasing number of vectors. In each layer $\mathcal{L}_b$, a directed edge from o_i to o_j indicates that o_j is stored as a neighbor of o_i; in this case, o_j is called an out-neighbor of o_i. The set of out-neighbors of o_i is denoted as $N_b[o_i]$, the cardinality of which is called the out-degree of o_i.

HNSW Construction. An HNSW can be constructed by constantly inserting objects (e.g., o) with the `InsertObject` method in Algorithm 1. To begin construction, we set the maximum out-degree M of each vertex and the maximum number of candidates ef_{con} proposed as the nearest neighbors of o. Next, we sample a random level $\mathcal{L}_a$, downwards from which the object o will be inserted. Before that, starting from $\mathcal{L}_{\max}$ (Lines 3–4), we find the entry point for each layer above $\mathcal{L}_a$ by applying a `SearchLayer` procedure with $ef = 1$, which narrows the subsequent search space by providing a good nearest neighbor as the entry point. The details of `SearchLayer` are omitted and can be found in [7]. For each layer $\mathcal{L}_b$ not above $\mathcal{L}_a$, we search for ef_{con} nearest neighbors of o, denoted as $\mathcal{W}$ (Line 6), from which at most M neighbors are selected to form the out-neighbors $N_b[o]$ (Line 7). For each out-neighbor $v \in N_b[o]$, we create an edge between o and v. Moreover, in Lines 8-9, the existing edges in $N_b[v]$ are updated and pruned to maintain search efficiency. The procedure `PruneExistingEdges` uses the strategy of `SelectNeighbors` in [7]. Finally, the updated HNSW $\mathcal{H}$ is returned (Line 10). The expected number of edges created in HNSW is $O(nM(1+1/\ln(M)))$, and the term $1+1/\ln(M)$ is the expected number of layers an object appears in HNSW. M is usually 16-64, and thus the space cost of HNSW is $O(nM)$.

Search on HNSW. Given a query vector q, the search in HNSW starts from $\mathcal{L}_{\max}$ and proceeds downwards to the base layer $\mathcal{L}_0$. In each layer, HNSW applies the `SearchLayer` procedure to find the nearest neighbor of o, which serves as the entry point for the next layer. In the base layer $\mathcal{L}_0$, `SearchLayer` is called again to find ef nearest neighbors of q, from which the top-k are returned as the final result.

III. Related Works

A. General ANNS

The research on ANNS can be roughly categorized into the methods based on the tree structure, hashing, graphs, and quantization. The tree-based methods [25], [26] leverage hierarchical structures to separate data, enabling fast searches with approximate pruning. However, they are less effective in high dimensions. Hashing-based methods [27]–[29] map similar points to the same buckets by probabilistic hash functions, providing scalability for large datasets. Graph-based methods [24], [30]–[32], such as HNSW [7] and DiskANN [23], build proximity graphs for highly accurate and tunable searches. Quantization-based methods [33]–[36] compress data into compact codes to reduce memory usage and accelerate distance computations. Some works also apply hybrid ideas to solve ANNS, e.g., [30]. For a more detailed tutorial, the readers may refer to [37], [38].

B. Attribute Filtering ANNS

This formulation extends ANNS by requiring returned objects to satisfy additional attribute predicates. The Native Hybrid Query framework in [39] achieves up to 315× speedup. Gollapudi et al. propose Filtered-DiskANN to support categorical and numerical filters under the disk-based setting [40]. The HQI system [41] is designed for ANNS in knowledge graphs. In [42], the authors propose a reconfigurable index for multimedia datasets with significant speedups. ARKGraph in [43] supports range queries on vectors with numerical attributes, which is closely related to RFANNS. ACORN increases the number of edges in HNSW to handle low-selectivity queries with general predicates. Milvus supports attribute filtering through hybrid queries that combine vector similarity with metadata constraints [11].

C. Range Filtering ANNS (RFANNS)

Early works solve RFANNS by combining searching and filtering. For example, pre-filtering uses binary search to find in-range objects and then computes distances. It is efficient only for very small query ranges (e.g., $\leq$ 1024 objects). Post-filtering first retrieves candidate nearest neighbors via ANNS, then applies the range filter to obtain k in-range

neighbors. It becomes inefficient for selective queries with few in-range items. SuperPostfiltering [44] improves post-filtering by searching the shortest pre-built range that covers the query range. The covering range may be larger than the query range, causing extra distance computations on out-of-range objects.

Later works try to apply more complex indexes (such as HNSW, segment trees) to improve the efficiency. iRangeGraph [9] builds a segment tree over attribute values, attaching a graph index to each segment. For a query range, it consults $O(\log n)$ segments and constructs a dedicated graph on the fly. To process each object, the amortized time of iRangeGraph is $O(M + \log n)$, and the total space is $O(nM \log n)$. 2DSegmentGraph [10] extends HNSW by attaching attribute-related tags to each edge. For every edge, four tags (l, r, b, e) are stored to check query range bounds. An edge is eligible if the query range falls into these intervals. This approach may create many edges per node, leading to $O(n^2 M)$ worst-case space and an average space of $O(nM \log n)$. UNIFY [18] builds a unified HSIG index for RFANNS over one scalar attribute. It combines segment-wise graph structures, skip-list connections, and compressed global HNSW edges to support hybrid, pre-, and post-filtering within a single index, with range-aware strategy selection. The most recent method called WoW [17] sorts scalar attributes with a weighted balanced tree for window boundaries, and maintains hierarchical window graphs. In each layer, neighbors are restricted to a symmetric attribute window. It selects a landing layer to balance filter checks and distance computations.

IV. Index Structure and Construction

An efficient RFANNS index should satisfy two requirements: search should be restricted to the query range, and the underlying proximity graph should remain navigable for high-recall search. A direct idea to solve RFANNS is partitioning the search space into many small segments and building indexes for them. This idea preserves range precision but impairs graph connectivity. Conversely, searching a larger segment can preserve connectivity but may incur unnecessary exploration.

To address the above dilemma, FRASP proposes to represent every query range with a constant number of segment indexes at its first-split level. Specifically, the index of FRASP is designed under four principles. (i) FRASP uses a first-split range representation to express the query range as two one-sided child subranges contained by a parent segment. Next, we construct an HNSW for each segment. As a result, each query only accesses three HNSWs. (ii) To construct the HNSWs for the two one-sided segments, FRASP inserts objects in attribute order and retains pruned edges with deletion tags. In this way, we can recover the HNSW that would have been constructed directly over the queried one-sided segment. (iii) The HNSW of a segment remains unchanged regardless of whether the segment serves as a child or parent, without materializing additional bridge indexes. Then, FRASP performs a unified search to traverse the three HNSWs with shared candidate and result queues. (iv) To support range-aware HNSW initialization, FRASP maintains lightweight auxiliary arrays for each segment HNSW, so the search can start from an in-range upper-layer entry point without scanning layers.

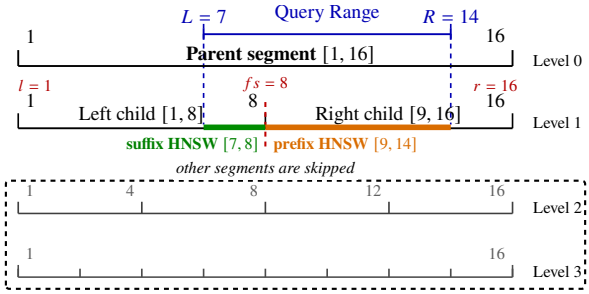


Fig. 1. Suffix/Prefix structure and the first-split position of query $[7, 14]$ in a segment tree.

A. First-Split Range Strategy

We adopt the segment tree to handle the ranges, which is a classic binary tree structure over the numerical range $[1, n]$. Each node represents a segment and is recursively split at its midpoint. Figure 1 shows an example of a segment tree over $[1, 16]$. Although many segments in the tree may intersect a query range, FRASP does not decompose the range into all canonical nodes. Instead, it identifies the first level of the segment tree at which the query range crosses a midpoint when traversing from the root. Given a query range $[L, R]$, let $[l, r]$ be the *highest* segment-tree node whose midpoint $mid = \lfloor (l + r)/2 \rfloor$ splits the query range, i.e., $L \leq mid < R$. Then, we call $[l, r]$ the parent segment of the query range, $fs = mid$ the *first-split position*, and the level that contains $[l, r]$ the *first-split level*. The two children of the parent segment, namely $[l, fs]$ and $[fs+1, r]$, are called the *left child segment* and the *right child segment*, respectively. In analogy, the HNSW built on $[l, r]$ will be referred to as the *parent HNSW*, while the suffix HNSW of $[l, fs]$ and the prefix HNSW of $[fs + 1, r]$ will be referred to as the *child HNSWs*.

Accordingly, the query range $[L, R]$ is decomposed into a suffix range $[L, fs]$ contained in the left child segment and a prefix range $[fs + 1, R]$ contained in the right child segment. This decomposition reduces arbitrary range search to two one-sided subrange searches and one parent segment for connectivity.

***Example* 1.** *Figure 1 illustrates the first-split range representation. The segment tree is built over the attribute domain* $[1, 16]$*, and the query range is* $[7, 14]$*. Since* $[7, 14]$ *crosses the midpoint* $fs = 8$ *of the root segment* $[1, 16]$*, the root is the parent segment, and* $fs = 8$ *is the first-split position.*

Accordingly, the query range is split into two one-sided ranges: the suffix range $[7, 8]$ *in the left child* $[1, 8]$*, and the prefix range* $[9, 14]$ *in the right child* $[9, 16]$*. Therefore,* FRASP *only needs to access the suffix HNSW restricted to* $[7, 8]$*, the prefix HNSW restricted to* $[9, 14]$*, and the parent HNSW on* $[1, 16]$ *restricted to* $[7, 14]$*. Other lower-level segments are skipped, even if they intersect the query range.*

B. Suffix and Prefix HNSW Indexing

Note that dedicated techniques are unnecessary for very small ranges (e.g., containing fewer than $B = 1024$ objects), as such queries can be efficiently handled by pre-filtering. Thus, for the segment tree, we truncate levels with segment size $\leq B/2$, resulting in $O(n/B)$ nodes and height $O(\log(n/B))$. This suffices for all query ranges larger than B, since our design only requires searching the three segments at the first-split level, and any such query must have a first-split segment size $> B/2$.

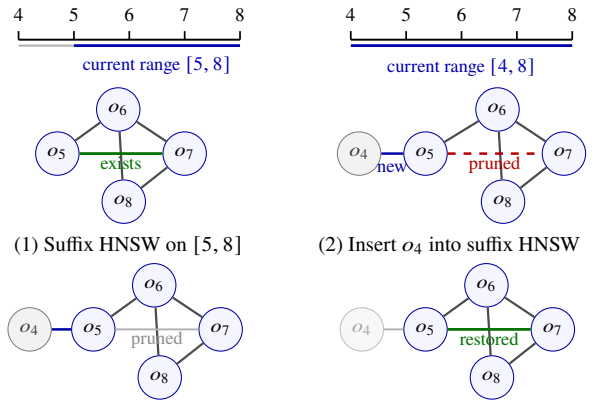
For each segment, we construct a specialized HNSW. If the segment is a left child of the segment tree, we construct a suffix HNSW for it. Otherwise, we construct a prefix HNSW for it. The root segment employs a prefix HNSW by default. Next, we detail the construction of the suffix HNSW for a left child segment $[l, r]$, and the construction of a prefix HNSW for a right child is analogous.

Suffix HNSW Construction. The suffix HNSW consists of (i) an HNSW graph index $\mathcal{H}_{\text{sfx}}$ and (ii) a pruned edge index N^{del} that stores the edges pruned during graph construction. Given a segment $[l, r]$ which is a left child in the segment tree, $\mathcal{H}_{\text{sfx}}$ is constructed by inserting objects in suffix order, i.e., from r down to l in descending order of attribute values. During the insertion of each object o_i, the pruned edges are recorded in N^{del} with a *deletion tag* $= i$ for each pruned edge, indicating that the edge was deleted during the insertion of o_i.

Algorithm 2 describes the construction of the suffix HNSW for $[l, r]$. After initializing empty $\mathcal{H}_{\text{sfx}}$ and N^{del}, we also create an array A to record the topmost layer where each object starts being inserted, which will be used in Section IV-C. Objects (e.g., o_i) are inserted in suffix order from r down to l (Line 3). In Line 4, similar to Lines 1-4 in Algorithm 1, we sample a random layer $\mathcal{L}_a$ and obtain the entry point ep_a. Downwards from $\mathcal{L}_a$, we start inserting o_i. The topmost layer $\mathcal{L}_a$ of inserting o_i is recorded in $A[i]$ at Line 5. For each layer $\mathcal{L}_b$ from $\mathcal{L}_a$ to $\mathcal{L}_0$, by Lines 6 to 10 we create new edges for o_i (i.e., $N_b[o_i]$), and prune existing edges of every neighbor o_j in $N_b[o_i]$ using the new edge (o_i, o_j). The key point is that, before pruning, we copy the existing neighbors of o_j into $N'_b[o_j]$ (Line 9). The difference between $N'_b[o_j]$ and the updated neighbors $N_b[o_j]$ identifies the pruned edges (o_j, o_t) (Line 11), which are stored in $N^{\text{del}}_b[o_j]$ with a deletion tag i (Line 12). The setting $tag = i$ indicates that the edge (o_j, o_t) was deleted during the insertion of o_i into $\mathcal{H}_{\text{sfx}}$. If the edge (o_j, o_t) lies within the suffix subrange $[L, r]$ (i.e., o_j and o_t are both in $[L, r]$) and has $tag = i < L$, then the edge was deleted due to an insertion outside the query range. Thus, the edge remains valid for the range $[L, r]$.

Lemma 1 states that the in-range edges in $\mathcal{H}_{\text{sfx}}$, combined with the in-range pruned edges in N^{del} where $tag = i < L$, collectively form an HNSW equivalent to one constructed directly over $[L, r]$ using the same M and ef_{con} parameters.

Lemma 1. *Given a suffix HNSW for segment $[l, r]$, consisting of $\mathcal{H}_{sfx}$ and N^{del}, for any suffix query range $[L, r]$ ($l \leq L \leq r$), the HNSW induced by objects in $[L, r]$ is equivalent to the HNSW that would be constructed directly over $[L, r]$ with the same insertion order, M, and ef_{con}.*



(1) Suffix HNSW on $[5, 8]$ (2) Insert o_4 into suffix HNSW

(3a) Query $[4, 8]$: current HNSW (3b) Query $[5, 8]$: virtual HNSW

Fig. 2. An example on inserting objects to a suffix HNSW.

Proof sketch. Fix the same sampled levels and construction rules. After inserting o_L, the suffix HNSW is the graph built on $[L, r]$. Later insertions with $i < L$ can prune in-range edges but cannot create new edges. Thus, restoring the recorded in-range edges with tags $i < L$ recovers that graph. □

***Example* 2.** *Figure 2 illustrates how a suffix HNSW is maintained and reused for different suffix ranges. Figure 2(1) first depicts the suffix HNSW for range $[5, 8]$, where objects o_8, o_7, o_6, o_5 have been inserted in descending attribute order.*

Figure 2(2) then shows the effect of inserting o_4. The maintained range is extended to $[4, 8]$; a new edge (o_4, o_5) is added; and the existing edge (o_5, o_7) is pruned by the standard HNSW neighbor selection due to the degree constraint. `FRASP` stores this removed edge in N^{del} with deletion tag 4, indicating that the edge is removed when o_4 is inserted.

The two cases in Figures 2(3a) and 2(3b) show how the same maintained suffix HNSW serves different query ranges. For query range $[4, 8]$, o_4 is in range, so its insertion is valid and the current graph is used directly. For query range $[5, 8]$, o_4 is out of range; hence the pruning effect caused by o_4 should be undone. Since the stored tag on the pruned edge satisfies $4 < L = 5$, `FRASP` restores (o_5, o_7). This allows `FRASP` to represent multiple suffix ranges with one maintained suffix HNSW and tagged pruned edges, rather than materializing an independent graph for each range.

Prefix HNSW Construction. The construction of the prefix HNSW for a right child segment $[l, r]$ is analogous to the suffix HNSW, except that objects are inserted in ascending order from l to r, as shown in Algorithm 3. In the prefix HNSW, a pruned edge (o_j, o_t) with $tag = i$ indicates it was deleted during the insertion of o_i. If an edge (o_j, o_t) lies within the prefix query range $[l, R]$ and has $tag = i > R$, then the edge was deleted due to an insertion outside the query range. Therefore, the edge remains valid for the query range $[l, R]$. The prefix HNSW satisfies a symmetric property to Lemma 1.

Algorithm 2: `BuildSuffixHNSW`($[l, r]$)

Input: Segment $[l, r]$, objects $\{o_l, \ldots, o_r\}$, M, ef_{con}
Output: $\mathcal{H}_{\text{sfx}}$, N^{del} for $[l, r]$ and A

1 Initialize empty $\mathcal{H}_{\text{sfx}}$ and N^{del};
2 Initialize A as a zero-array for indices in $[l, r]$;
3 **for** *i from r down to l* **do**
4 $\quad \mathcal{L}_a, ep_a \leftarrow$Same as Lines 1-4 in Algorithm 1;
5 $\quad A[i] \leftarrow a$; `// Record the topmost layer of` o_i
6 $\quad$ **for** *$\mathcal{L}_b$ from $\mathcal{L}_a$ to $\mathcal{L}_0$* **do**
7 $\quad\quad N_b[o_i] \leftarrow$ Same as Lines 6-7 in Algorithm 1;
8 $\quad\quad$ **foreach** $o_j \in N_b[o_i]$ **do**
9 $\quad\quad\quad N'_b[o_j] \leftarrow N_b[o_j]$;
10 $\quad\quad\quad N_b[o_j] \leftarrow$ `PruneExistingEdges` $(N_b[o_j] \cup \{o_i\})$;
11 $\quad\quad\quad$ **foreach** $o_t \in N'_b[o_j] \setminus N_b[o_j]$ **do**
12 $\quad\quad\quad\quad N_b^{\text{del}}[o_j]$.add$((o_t, tag = i))$;
13 **return** $\mathcal{H}_{\text{sfx}}$, N^{del}, A;

Algorithm 3: `BuildPrefixHNSW`($[l, r]$)

Input: Segment $[l, r]$, objects $\{o_l, \ldots, o_r\}$, M, ef_{con}
Output: $\mathcal{H}_{\text{pfx}}$, N^{del} for $[l, r]$ and A

1 Initialize empty $\mathcal{H}_{\text{pfx}}$ and N^{del};
2 Initialize A as a zero-array for indices in $[l, r]$;
3 **for** *i from l to r* **do**
$\quad$ `// Same as Algorithm 2`

Discussion. While the segment graph in SeRF [10] also fixes insertion orders to support one-sided query ranges, `FRASP` adopts different designs to handle arbitrary query ranges. Specifically, SeRF extends to 2DSegmentGraph for arbitrary query ranges by embedding four attribute-related tags per edge for bound checking. In contrast, `FRASP` only requires a single tag per pruned edge. Moreover, `FRASP` efficiently supports arbitrary ranges by enhancing a segment tree with prefix and suffix HNSWs. The performance improvements in experiments also validate the effectiveness of our design.

C. Prefix/Suffix Max Array Indexes

Given a query range $[L, R]$, the top layers of an HNSW may contain no object in $[L, R]$ due to their sparsity. Thus, we must identify the highest layer containing at least one in-range object (i.e., the *topmost query-relevant layer* $\mathcal{L}_{\text{qmax}}$) and select such an object as the *query-relevant entry point* ep_{qmax}.

Recall that for any $[L, R]$ within a parent segment $[l, r]$, the midpoint mid is the first-split position fs. When building a specialized HNSW for $[l, r]$, we also recorded the highest layer where each object is inserted in an array A (see Line 5 in Algorithm 2). The midpoint mid splits A into two parts, $A[l : mid]$ and $A[mid+1 : r]$, corresponding to the suffix and prefix subranges of the query range $[L, R]$. Leveraging this information, we build a max array index for each segment, to identify $\mathcal{L}_{\text{qmax}}$ and ep_{qmax} for $[L, R]$ in the parent HNSW through simple array lookups during search.

Specifically, with $A[l : mid]$, we construct a *suffix-Max array* Z_1 by a linear scan from mid down to l in A. Here, $Z_1[i]$ stores the index of the object with the highest layer among all objects in the suffix range $[i, mid]$, where i is an index in $[l, mid]$. Thus, the topmost query-relevant layer for the suffix query range $[L, mid]$ is $A[Z_1[L]]$, and the query-relevant entry point is $Z_1[L]$. Similarly, for $A[mid+1 : r]$, we build a *prefix-Max array* Z_2 by a linear scan from $mid + 1$ up to r in A. Here, $Z_2[i]$ stores the index of the object with the highest layer among all objects in the prefix range $[mid+1, i]$, where i is an index in $[mid + 1, r]$. Thus, the topmost query-relevant layer for the prefix query range $[mid + 1, R]$ is $A[Z_2[R]]$, and the query-relevant entry point is $Z_2[R]$.

With suffix-Max and prefix-Max arrays, we determine $\mathcal{L}_{\text{qmax}}$ and ep_{qmax} by Eq. (1). That is, during query processing, we compare the entry points identified by the suffix-Max and prefix-Max arrays, and select the one with the higher layer as the final $\mathcal{L}_{\text{qmax}}$ and ep_{qmax} for $[L, R]$ in the parent HNSW.

$$\mathcal{L}_{\text{qmax}} = \max\left(A[Z_1[L]], A[Z_2[R]]\right)$$
$$ep_{\text{qmax}} = \begin{cases} Z_1[L], & \text{if } A[Z_1[L]] > A[Z_2[R]] \\ Z_2[R], & \text{otherwise} \end{cases} \quad (1)$$

Algorithm 4 presents the pseudo code of constructing the suffix-Max and prefix-Max arrays Z_1 and Z_2 for the parent segment $[l, r]$ from A. After computing mid and initializing the two arrays in Lines 1–2, Lines 3–8 build Z_1 by scanning from mid down to l. The base case is $Z_1[mid] = mid$. For each $i \in [l, mid-1]$ scanned in descending order, we compare $A[i]$ with $A[Z_1[i + 1]]$. If $A[i]$ is larger, we set $Z_1[i] = i$; otherwise, we inherit the previous argmax, i.e., $Z_1[i] = Z_1[i+1]$. Hence, $Z_1[i]$ stores the index of the highest-layer object in the suffix range $[i, mid]$.

Lines 9–14 build Z_2 symmetrically by scanning from $mid+1$ up to r. The base case is $Z_2[mid+1] = mid+1$. For each $i \in [mid+2, r]$, we compare $A[i]$ with $A[Z_2[i-1]]$. If $A[i]$ is larger, we set $Z_2[i] = i$; otherwise, we set $Z_2[i] = Z_2[i-1]$. Thus, $Z_2[i]$ stores the index of the highest-layer object in the prefix range $[mid + 1, i]$.

Algorithm 4: `BuildMaxArray`

Input: Array A for parent segment $[l, r]$
Output: suffix-Max array Z_1 , prefix-Max array Z_2

1 $mid \leftarrow \lfloor (l + r)/2 \rfloor$;
2 Initialize Z_1 for $[l, mid]$, Z_2 for $[mid + 1, r]$ as zero-arrays;
$\quad$ `// Build suffix-Max array`
3 $Z_1[mid] \leftarrow mid$;
4 **for** *i from $mid - 1$ down to l* **do**
5 $\quad$ **if** $A[i] > A[Z_1[i + 1]]$ **then**
6 $\quad\quad Z_1[i] \leftarrow i$;
7 $\quad$ **else**
8 $\quad\quad Z_1[i] \leftarrow Z_1[i + 1]$;
$\quad$ `// Build prefix-Max array`
9 $Z_2[mid + 1] \leftarrow mid + 1$;
10 **for** *i from $mid + 2$ to r* **do**
11 $\quad$ **if** $A[i] > A[Z_2[i - 1]]$ **then**
12 $\quad\quad Z_2[i] \leftarrow i$;
13 $\quad$ **else**
14 $\quad\quad Z_2[i] \leftarrow Z_2[i - 1]$;
15 **return** Z_1, Z_2;

Algorithm 5: FRASP Index Construction

Input: Objects $\mathcal{D}$, HNSW parameters M, ef_{con}, B
Output: FRASP-Index

1 Build a segment tree over $[1, n]$ with B;
2 **foreach** *segment* $[l, r]$ *in the segment tree* **do**
3 **if** $[l, r]$ *is a left child* **then**
4 $\mathcal{H}_{\text{sfx}}, N^{\text{del}}, A \leftarrow$ BuildSuffixHNSW$([l, r], M, ef_{\text{con}})$;
5 **else if** $[l, r]$ *is a right child or the root segment* **then**
6 $\mathcal{H}_{\text{pfx}}, N^{\text{del}}, A \leftarrow$ BuildPrefixHNSW$([l, r], M, ef_{\text{con}})$;
7 **if** $[l, r]$ *is not a leaf* **then**
  // It can serve as a parent segment
8 $Z_1, Z_2 \leftarrow$ BuildMaxArray(A);

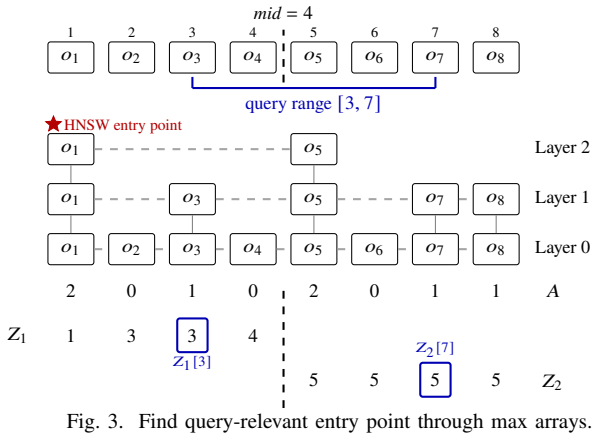


Fig. 3. Find query-relevant entry point through max arrays.

***Example* 3.** *Figure 3 illustrates how* FRASP *locates an in-range entry point in the parent HNSW. For query range* $[3, 7]$ *in parent segment* $[1, 8]$*, the midpoint* $mid = 4$ *splits it into* $[3, 4]$ *and* $[5, 7]$*.* Z_1 *and* Z_2 *shown in the figure correspond to the suffix-Max array and the prefix-Max array, respectively.*

By array lookup, $Z_1[3] = 3$ *gives object* o_3 *with layer* $A[3] = 1$*, while* $Z_2[7] = 5$ *gives object* o_5 *with layer* $A[5] = 2$*. Since* $A[5] > A[3]$*, Eq.* (1) *sets* $ep_{qmax} = o_5$ *and* $\mathcal{L}_{qmax} = \mathcal{L}_2$*. Thus,* FRASP *starts the upper-layer search from the highest in-range object instead of the general HNSW entry point.*

D. Overall Indexing and Analysis

The overall procedure of FRASP constructing the index is provided in Algorithm 5. We first construct a segment tree over the input data range $[1, n]$ (Line 1), which is a standard procedure and thus the details are omitted. For each segment in this tree, if it is a left child, we build a suffix HNSW $\mathcal{H}_{\text{sfx}}$ with N^{del} pruned edges (Lines 3-4). If the segment is a right child or the root, it constructs a prefix HNSW $\mathcal{H}_{\text{pfx}}$ with N^{del} pruned edges (Lines 5-6). For non-leaf segments that can serve as parents during query processing, we build suffix-max and prefix-max arrays Z_1 and Z_2 (Lines 7-8).

Construction Time of Prefix/Suffix HNSW. The time complexity of constructing a prefix or suffix HNSW is equivalent to that of a standard HNSW. For a standard HNSW with n objects and parameter M, the total number of edges is $O(nM)$. During construction, the procedure of pruning edges may delete up to M edges, leading to a total of $O(nM)$ pruned edges for a single HNSW in the worst case. Thus, the overall space complexity of a prefix or suffix HNSW, including pruned edges, remains $O(nM)$, consistent with the original HNSW. Empirically, the number of pruned edges is often a small fraction of the total edges, as shown in the experiments.

Space Complexity. For a standard HNSW with n objects and parameter M, the number of edges is $O(nM)$. More precisely, the expected number of edges is $O(nM(1+1/\ln(M)))$, where the term $M(1+1/\ln(M))$ accounts for the expected number of edges per object across all layers in an HNSW [7]. Usually M is 16 to 64, meaning that $1/\ln(M) < 1$, thus it is reduced to $O(nM)$. Since objects in different segments of the same level in the segment tree are disjoint, the total edge complexity of HNSWs in the same level is $O(nM)$. The total number of pruned edges for each level of the segment tree is also $O(nM)$ in the worst case. As there are $O(\log(n/B))$ levels in the segment tree, the total number of edges in the whole FRASP index is $O(nM\log(n/B))$, reduced to $O(nM\log n)$ if B is a constant. For every segment, we build arrays A, Z_1, and Z_2, all of which together use $O(n\log n)$ space. Therefore, the total space complexity of FRASP is $O(nM\log n)$.

Runtime Analysis. The main computational cost of HNSW indexing comes from searching through its layers. Although it has been shown that no algorithm can guarantee sub-linear time complexity for nearest neighbor search [28], [45], [46], graph-based indexing methods often exhibit sub-linear search performance in practice on real-world datasets [9]. Therefore, theoretical asymptotic complexity may not fully capture practical performance. Following the assumption in [9], we focus on analyzing the additional overhead introduced by our index construction compared to building a single HNSW over the entire object set $\mathcal{D}$.

Let $h(n)$ denote the time complexity to build a single HNSW, and $f(n)$ denote the time complexity to search during edge creation, where n is the number of objects. Then, we derive the time complexity of generating the indexes by FRASP in Theorem 4.

Theorem 4 (Indexing Time Complexity)**.** *The time complexity of* FRASP *constructing indexes is* $h(n)\log(n/B)$*. For the search process during edge creation, the runtime of* FRASP *is bounded by* $h(n)\cdot\frac{1-(1/2^c)^{\log(n/B)+1}}{1-1/2^c}$*, whose maximum is* $\frac{1}{1-\sqrt{2}/2}$ *obtained at* $c=\frac{1}{2}$ *where* $\frac{1}{2}<c\leq 1$*.*

Proof sketch. Note that the SearchLayer operation takes more time on larger HNSWs. Then, $\forall x, y \in \mathbb{N}$, s.t. $x + y = n$, we have $h(x) + h(y) \leq h(n)$. W.o.l.g., we assume n is divisible by B and 2^i. So the segment tree has $t = \log(n/B)+1$ layers. In each layer i, we have 2^i elements whose size is $n/2^i$. Thus, the FRASP index time is $\sum_{i=0}^{t-1}(2^i)h(\frac{n}{2^i}) \leq \sum_{i=0}^{t-1} h(n) = h(n)(\log(n/B)+1)$. Let $f(n)$=$O(n^c)$ be the time of SearchLayer on NSW graph [9]. The expected number of objects in layer $\mathcal{L}_a$ is $ne^{-a\ln M}$ [7]. By summing the searching time on every layer, we derive the time complexity of applying SearchLayer $g(n) = O(n^c)$. Naturally, we have $h(n) = O(n^{c+1})$.

Then, the index time that only considers the SearchLayer process is $\sum_{i=0}^{t-1} 2^i\, h\left(\frac{n}{2^i}\right) = h(n) \cdot \frac{1-(1/2^c)^{\log(n/B)+1}}{1-1/2^c}$. □

V. Query Processing

With the indexes built, we are ready to detail the processing of queries. Given a query vector q and range $[L, R]$, we first locate the first-split position $fs = mid = \lfloor (l+r)/2 \rfloor$ in the segment tree, where $[l, r]$ is the minimal segment containing $[L, R]$ (also, the parent segment). The query range is then split into the suffix range $[L, mid]$ and prefix range $[mid + 1, R]$. The suffix HNSW $\mathcal{H}_{\text{sfx}}$ supports $[L, mid]$; the prefix HNSW $\mathcal{H}_{\text{pfx}}$ supports $[mid + 1, R]$; and the parent HNSW $\mathcal{H}_{\text{p}}$ over $[l, r]$ provides inter-segment connectivity. In a special case where the query range $[L, R]$ has $R = mid$ (resp. $L = mid+1$), we only need to search one HNSW, the suffix HNSW $\mathcal{H}_{\text{sfx}}$ (resp. prefix HNSW $\mathcal{H}_{\text{pfx}}$). Next, we only focus on the general case where $L \le mid$ and $R > mid$.

For efficient query processing, FRASP employs three ideas. (i) A *single unified search process* is performed over the base layers of $\mathcal{H}_{\text{sfx}}$, $\mathcal{H}_{\text{pfx}}$, and $\mathcal{H}_{\text{p}}$, using a shared candidate priority queue $\mathcal{C}$ and result priority queue $\mathcal{R}$, with a global lower bound $\mathcal{R}$.maxDist for pruning and early termination. (ii) The search is strictly *in-range*, starting from entry points found via the suffix-max and prefix-max arrays in Section IV-C, and explores only objects within $[L, R]$, ensuring efficiency even for small overlaps with the segments. (iii) Any edge between objects o_i and o_j may appear in both $\mathcal{H}_{\text{p}}$ and either $\mathcal{H}_{\text{sfx}}$ or $\mathcal{H}_{\text{pfx}}$, but is *traversed at most once* during the search, avoiding redundant computations.

The process to perform the query by FRASP is presented in Algorithm 6. We first find the first-split position $fs = mid$ of query range $[L, R]$ and the parent segment $[l, r]$, by applying a routine called FindFirstSplitPosition, which utilizes a simple binary search in the segment tree. Thus, its pseudocode is omitted here but deferred to Algorithm 8 in our technical report [20]. Next, we derive the left child $[l, mid]$, right child $[mid + 1, r]$, and parent $[l, r]$ segments associated with their HNSWs ($\mathcal{H}_{\text{sfx}}, \mathcal{H}_{\text{pfx}}$, and $\mathcal{H}_{\text{p}}$, resp.) in Line 2. In Lines 3-4, for the parent HNSW $\mathcal{H}_{\text{p}}$, we apply Eq. (1) to determine its topmost query-relevant layer and the query-relevant entry point, which are passed to a routine FindBaseEP in Line 4, which locates an in-range nearest neighbor in $\mathcal{L}_1$ as the base-layer entry point ep_{p}. In Line 5, we initialize a min-priority queue $\mathcal{C}$ to store possible nearest neighbors, i.e., candidates, and a max-priority queue $\mathcal{R}$ to store the actual nearest neighbors, by initializing them with the entry point of the base layer ep_{p}[2], along with their distances to q.

The details of FindBaseEP are presented in Algorithm 7. FindBaseEP performs a search on $\mathcal{H}_{\text{p}}$ from $\mathcal{L}_{\text{qmax}}$ down to $\mathcal{L}_1$, updating ep to the closest in-range neighbor per layer using a flag *changed* to track improvements. It only considers in-range neighbors and valid pruned edges. The process terminates when no closer neighbor is found, returning the final ep as the base-layer entry point. FindBaseEP is efficient, incurring negligible overhead compared to the main base-layer search, as it only traverses the middle layers to locate a single in-range entry point.

[2]In the special case where the query range matches the boundary case, only ep_{sfx} or ep_{pfx} is used instead.

Algorithm 6: FRASP Query Processing

Input: Query vector q, query range $[L, R]$, number of neighbors k, candidate size ef
Output: k nearest neighbors to q within range $[L, R]$

1 $fs = mid, l, r \leftarrow$ FindFirstSplitPosition($[L, R]$);
2 $\mathcal{H}_{\text{sfx}}, \mathcal{H}_{\text{pfx}}, \mathcal{H}_{\text{p}} \leftarrow$ HNSWs for $[l, mid], [mid + 1, r], [l, r]$;
3 $\mathcal{L}_{\text{qmax}}, ep_{\text{qmax}} \leftarrow$ Eq. (1) on Z_1, Z_2, A of $\mathcal{H}_{\text{p}}$;
4 $ep_{\text{p}} \leftarrow$ FindBaseEP($\mathcal{H}_{\text{p}}, q, [L, R], \mathcal{L}_{\text{qmax}}, ep_{\text{qmax}}$);
5 Initialize priority queues $\mathcal{C}$ and $\mathcal{R}$ with $(ep_{\text{p}}, \delta(q, ep_{\text{p}}))$;
// Shared search process on base layers
6 **while** $\mathcal{C}$ *is not empty* **do**
7 $o_c \leftarrow \mathcal{C}$.pop() // Pop the closest candidate
8 **if** $\delta(q, o_c) > \mathcal{R}$.maxDist **and** $|\mathcal{R}| \ge ef$ **then break**;
// Find the neighbors of o_c to be explored
9 Set $\mathcal{V} \leftarrow \emptyset$;
10 **foreach** $\mathcal{H}$ *of* $\mathcal{H}_{pfx}$, $\mathcal{H}_{sfx}$, *and* $\mathcal{H}_p$ *where* o_c *exists* **do**
11 **foreach** *neighbor* $o_j \in \mathcal{H}.N_0[o_c]$ **do**
12 **if** $L \le j \le R$ *and* o_j *is unvisited* **then**
13 $\mathcal{V}$.insert(o_j);
14 Mark o_j as visited;
15 **foreach** *neighbor* $(o_j, tag) \in \mathcal{H}.N_0^{del}[o_c]$ **do**
16 **if** *(($\mathcal{H}$ is suffix HNSW and $tag < L$) or ($\mathcal{H}$ is prefix HNSW and $tag > R$)) and* $L \le j \le R$ *and* o_j *is unvisited* **then**
17 $\mathcal{V}$.insert(o_j);
18 Mark o_j as visited;
19 **foreach** *neighbor* $o_j \in \mathcal{V}$ **do**
20 **if** $\delta(q, o_j) < \mathcal{R}$*.maxDist or* $\mathcal{R}$*.size* $< ef$ **then**
21 $\mathcal{C}$.insert($o_j, \delta(q, o_j)$);
22 $\mathcal{R}$.insert($o_j, \delta(q, o_j)$);
23 **if** $\mathcal{R}$.size $> ef$ **then** $\mathcal{R}$.pop();
24 **return** Top-k in $\mathcal{R}$;

Next, we conduct a single unified search process over the base layers of $\mathcal{H}_{\text{sfx}}$, $\mathcal{H}_{\text{pfx}}$, and $\mathcal{H}_{\text{p}}$, by iterating over the candidates in $\mathcal{C}$. Each time, we take out the candidate closest to q, denoted as o_c (Line 7). If the distance of the closest candidate to q is larger than $\mathcal{R}$.maxDist and $|\mathcal{R}| \ge ef$, we simply stop iterating through $\mathcal{C}$ (Line 8), just like HNSW [7]. Otherwise, we proceed and denote the HNSW among $\mathcal{H}_{\text{pfx}}$, $\mathcal{H}_{\text{sfx}}$, and $\mathcal{H}_{\text{p}}$ which contains o_c as $\mathcal{H}$.

In Lines 11–18, we try to insert the neighbors of o_c in $\mathcal{H}$ into $\mathcal{V}$. Specifically, for each neighbor o_j of o_c (i.e., $o_j \in N_0[o_c]$ in the base layer of $\mathcal{H}$), if o_j is in-range (i.e., $L \le j \le R$) and unvisited, we insert it into $\mathcal{V}$ and mark it as visited (Lines 11–14). To recover the deleted neighbors that should have been in the query range, for each deleted neighbor o_j in $N_0^{\text{del}}[o_c]$, if o_j is in-range, unvisited, and its deletion was caused by inserting an object outside the query range (i.e., $tag < L$ when $\mathcal{H}$ is a suffix HNSW or $tag > R$ when $\mathcal{H}$ is a prefix HNSW), we insert it into $\mathcal{V}$ and mark it as visited.

After obtaining the set $\mathcal{V}$, we iterate through each object o_j in $\mathcal{V}$ to update $\mathcal{C}$ and $\mathcal{R}$. If the distance from q to o_j (i.e., $\delta(q, o_j)$) is smaller than the maximum distance in $\mathcal{R}$, we

Algorithm 7: `FindBaseEP`

Input: HNSW $\mathcal{H}$, q, $[L, R]$, $\mathcal{L}_{\text{qmax}}$, ep_{qmax}
Output: base-layer entry point ep

```
1  ep ← ep_qmax;
2  for L_b from L_qmax down to L_1 do
3      changed ← true;
4      while changed do
5          changed ← false;
6          foreach neighbor o_j ∈ H.N_b[ep] do
7              if L ≤ j ≤ R and δ(q, o_j) < δ(q, ep) then
8                  ep ← o_j; changed ← true;
9          foreach neighbor o_j ∈ H.N_b^del[ep] do
10             if L ≤ j ≤ R and ((H is suffix HNSW and
                 neighbor[o_j].tag < L) or (H is prefix HNSW
                 and neighbor[o_j].tag > R)) then
11                 if δ(q, o_j) < δ(q, ep) then
12                     ep ← o_j; changed ← true;
13 return entry point ep for base layer;
```

insert o_j into both $\mathcal{C}$ and $\mathcal{R}$. After the insertion, if the size of $\mathcal{R}$ exceeds ef, we pop an object from $\mathcal{R}$ to keep its size no larger than ef (Lines 21–23).

The above iteration process goes on, until the candidate queue is empty. When the iteration stops, we return the top-k nearest neighbors in $\mathcal{R}$ (Line 24).

Time Complexity. The asymptotic query complexity is not a very informative measure for comparing graph-based methods, since their worst-case runtimes are typically linear to n, while their practical performance differs significantly, mainly determined by the number of objects processed during search. Thus, like [9], we analyze the time to process each object. Theorem 5 shows `FRASP` requires an expected $O(M)$ time per processed object.

Theorem 5. *The expected time complexity to process the edges of an object in* `FRASP` *is* $O(M)$.

Proof sketch. In a single HNSW, the expected out-degree of an object is $M + \frac{1}{\ln M}M = O(M)$ [7]. The average degree of N_b^{del} is at most $O(M)$. Since at most 3 HNSW graphs are used in one query and the two child HNSWs are disjoint, each object appears in at most two HNSWs. Thus, the number of explored edges per object remains $O(M)$. □

VI. Experiments

All methods are implemented in C++ and compiled with GCC 11.4.0 using -O3 and -march=native. Experiments are conducted on an Intel Xeon Platinum 8370C (2.80GHz, 384GB RAM). All search performance results use a single thread. Code is available at https://github.com/wjy-yy/FRASP.

A. Experimental Settings

Datasets. We use five widely adopted benchmarking vector datasets [9], [10], [44], [47]: WIT [48] (2048-dim ResNet50 image embeddings), YT-rgb [49] (1024-dim YouTube8M video features [50]), Sift [51] (128-dim image features), Gist [51] (960-dim image features), and Deep10M [52] (96-dim deep descriptors from the GoogLeNet). Attributes are: image size for WIT, number of likes for YT-rgb [9], random integers in $[1, 10^4]$ for Sift and Gist, id for Deep10M [52]. The tested datasets have different skewness measured by Fisher-Pearson coefficient. For example, the skewness of WIT is 4.15, while that of YT-rgb is 237.5. Each dataset contains one million objects except Deep10M, which contains 10^7 objects. One thousand vectors are sampled as queries, following [9].

TABLE II
Memory Footprint

	WIT	Sift	Gist	YT-rgb	Deep10M
Dimension	2048	128	960	1024	96
Raw Vectors	7.7	0.47	3.58	3.9	3.62
`FRASP`	9.13	2.24	5.01	4.50	26.23
iRangeGraph	12.87	1.97	8.81	9.05	26.80
SuperPostfiltering	28.81	7.34	16.64	17.36	91.90
SeRF	7.94	0.81	3.87	4.18	7.70
Milvus	8.41	0.78	4.05	4.31	7.22
ACORN	9.39	2.23	5.33	5.57	14.20
UNIFY	16.33	2.03	8.23	8.71	17.91
WoW	8.41	1.25	4.36	4.59	12.56

TABLE III
Number of pruned edges and total edges in the index.

Dataset	WIT	Sift	Gist	YT-rgb	Deep10M
Pruned Edges ($\times 10^6$)	33.20	49.15	30.10	2.23	926.73
Total Edges ($\times 10^6$)	176.50	216.18	167.60	57.30	3471.30
Ratio (%)	18.8	22.7	18.0	3.8	26.70

TABLE IV
Time breakdown of all queries in seconds.

Dataset	WIT	Sift	Gist	YT-rgb	Deep10M
Entry points	0.032	0.006	0.020	0.028	0.009
Dist computation	5.391	1.221	3.919	6.790	1.915
Graph traversal	1.249	1.028	1.332	1.658	1.600
Queue management	0.311	0.278	0.330	0.352	0.313
Total query time	6.984	2.534	5.601	8.828	3.837

Query Workloads. We evaluate different query workloads based on the range filter size, defined as the fraction of data objects covered by the query range [9]. Specifically, a range fraction of 2^{-i} means the query range covers $n/2^i$ ($i \in \{2, 4, 6, 8\}$) data objects. In the experiments, we first consider a mixed workload where the query vectors are randomly partitioned into 10 subsets. For the i-th subset ($i \in [0, 9]$), each query vector is assigned a random range with fraction 2^{-i}. These 10 subsets together form the mixed workload. Second, we consider fixed workloads, where all query vectors share the same range fraction. By varying i, we evaluate performance under large, medium, and small query ranges.

Evaluation Metrics. The efficiency of algorithms is measured by *queries per second (QPS)*, i.e., the number of queries processed per second [9], [47]. The accuracy of algorithms is measured by three metrics: *recall*, *distance relative error (DRE)*, and *total distance ratio (TDR)* [53]. Let Φ^* be the ground-truth k nearest neighbors and Φ the approximate results. Then, the recall is $|\Phi \cap \Phi^*|/|\Phi^*|$. DRE is computed as $\frac{1}{k}\sum_{i=1}^{k}[\delta(o_i, q) - \delta(o_i^*, q)]/\delta(o_i^*, q)$, where o_i is the i-th ANN and o_i^* the i-th NN. TDR is given by $\sum_{i=1}^{k}\delta(o_i, q)/\sum_{i=1}^{k}\delta(o_i^*, q)$.

Methods and Settings. We compare `FRASP` with iRange-

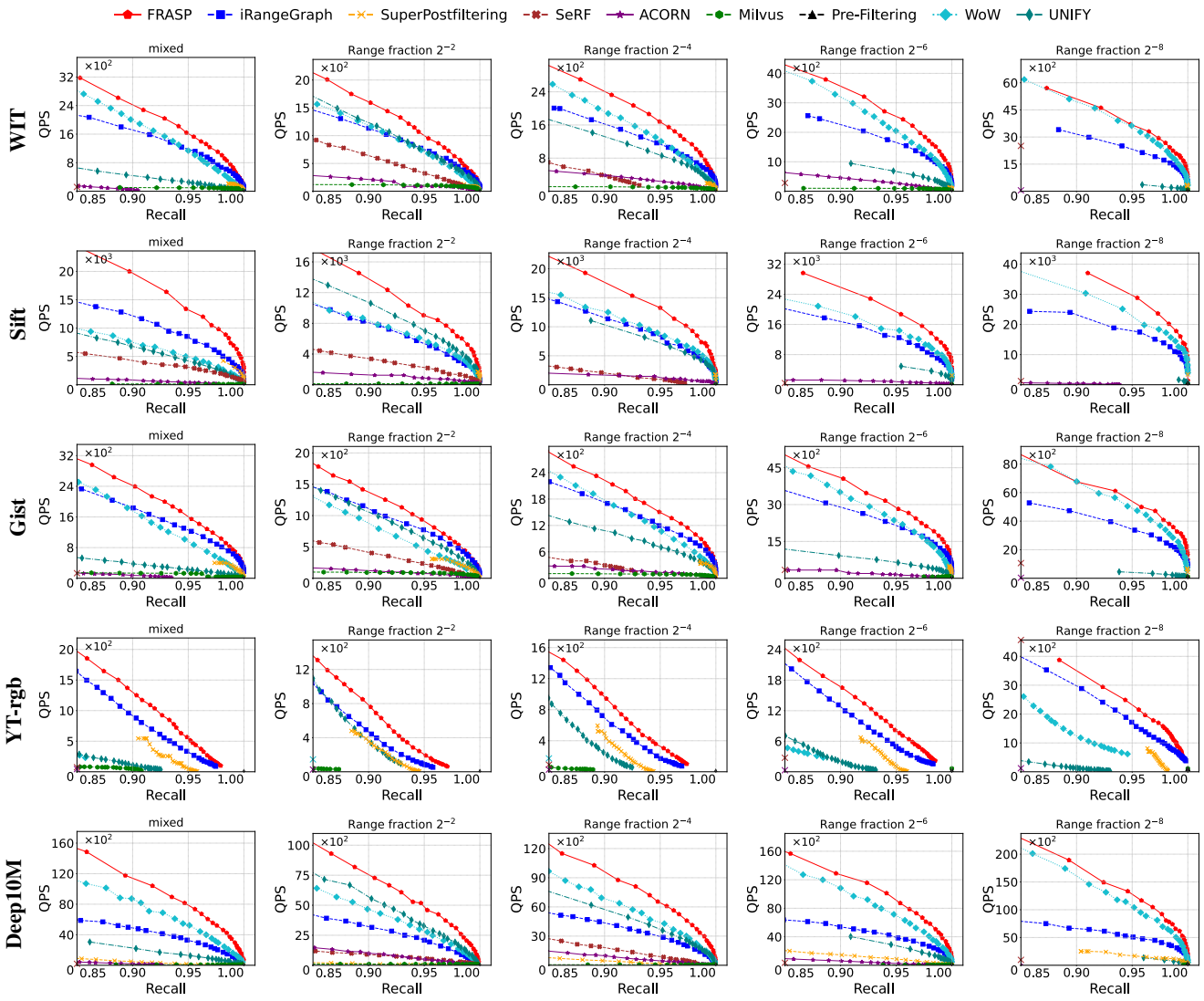


Fig. 4. QPS-Recall on WIT, Sift, Gist, YT-rgb, and Deep10M (top to bottom; mixed and range-fraction workloads left to right).

Graph [9], WoW [17], UNIFY [18], SuperPostfiltering [44], SeRF [10], ACORN [16], Milvus [11], and Pre-filtering, as reviewed in Section III. For all methods, $ef_{\text{con}} = 400$ unless otherwise specified. For all graph-based methods, we vary ef to adjust the QPS-Recall trade-off during query processing.

For FRASP, we set $M = 64$ for YT-rgb and $M = 16$ for the rest datasets, $B = 2^{10}$ for the first four datasets and $B = 2^{14}$ for Deep10M. M is also varied to study its impact on FRASP.

For the baselines, we use their recommended or default parameters whenever available. Notably, Pre-filtering does not need parameter setting. iRangeGraph only recommended setting $m = 64$ for WIT and YT-rgb. Then, for Sift and Gist, we use grid search and set $M = 16$, and we also set $M = 16$ for Deep10M following [17]. For SuperPostfiltering and UNIFY, we adopt their suggested value of $ef_{con} = 500$.

B. Experimental Results

QPS vs. Recall. Figure 4 shows the QPS-Recall curves of all methods across all datasets. We observe that FRASP consistently achieves higher QPS and recall than all baselines across all datasets and query workloads, often by a substantial margin. Under the RFANNS setting, the performance differences (e.g., Figure 4 with range fraction 2^{-2}) among iRangeGraph, WoW, SeRF and SuperPostfiltering are already considered as significant, but FRASP delivers even larger improvements over them. For the mixed workload over Sift, at recall 0.9, FRASP achieves about 19,700 QPS—1.6× and 4.6× that of iRangeGraph and SeRF (12,200 and 4,300 QPS, respectively), 23× and 130× that of ACORN (840 QPS) and Milvus (150 QPS). SuperPostfiltering usually has relatively good performance but FRASP still outperforms it by a large margin. Pre-filtering always achieves recall 1.0 but with low QPS, often appearing as an isolated point in the plots. These results demonstrate that FRASP consistently delivers high recall and efficiency for RFANNS queries, validating the superiority of our index and query processing approach.

QPS vs. DRE/TDR. We present the results of QPS vs. DRE in Figure 5. Observe that FRASP consistently achieves the best

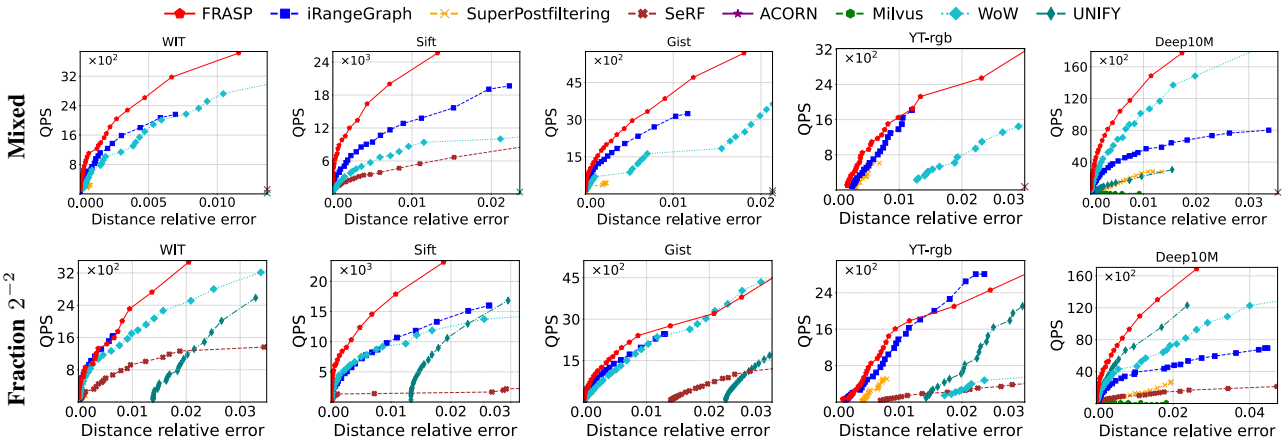


Fig. 5. QPS-Distance Relative Error on the mixed workload and range fraction 2^{-2}.

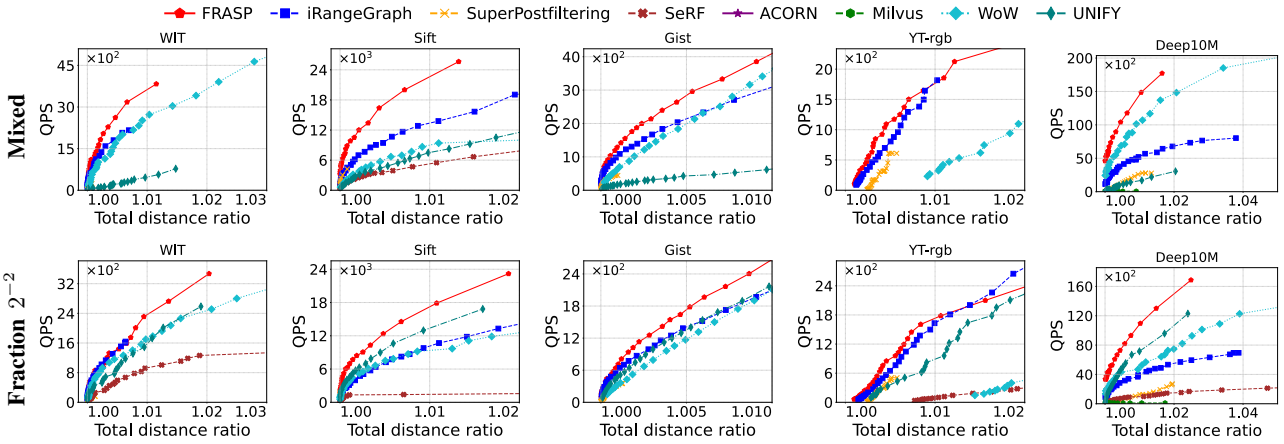


Fig. 6. QPS-Total Distance Ratio on the mixed workload and range fraction 2^{-2}.

QPS and relative error over existing methods. FRASP also consistently achieves a better QPS–TDR trade-off, as shown in Figure 6, where the total distance ratios are closer to the exact top-k results while maintaining higher throughput. This validates that FRASP can achieve low error of distances, via the technical designs proposed in Sections IV and V. Figure 7 reports the number of distance computations per query. The results under other workloads are deferred to our technical report [20] due to space limitations.

Index Cost. The cost of building FRASP index mainly includes the memory usage and the building time. We report the memory footprint (GB) for all methods except Pre-filtering (which does not require indexing) in Table II, including both the index and raw vectors. Among the top-performing methods, FRASP achieves a comparable or lower memory, validating the space complexity $O(nM \log n)$ in Section IV-D. Since only the indexes near the first-split position are considered, we can truncate the segment tree to consider the queries with ranges larger than B of interest. In contrast, iRangeGraph requires a complete segment tree, as it may need to access small segments for larger range queries. Thus, our index has a smaller memory footprint than iRangeGraph's on most datasets. For memory used during construction, FRASP is also smaller than iRangeGraph for most datasets, reducing by at most 22%. Table III shows the number of pruned edges and total edges (including all edges in N and all pruned edges stored in N^{del}, across all layers) in the index. Observe that the ratio of pruned edges to total edges is small, showing that storing the pruned edges is not a bottleneck in the index. For the time of building indexes, FRASP is comparable with iRangeGraph, saving up to 25% of the construction time.

Time Breakdown. As explained in Algorithm 6, we first find the in-range base layer entry points with minimal cost, and then perform the unified search. For all queries per dataset, we report the total query time, entry point finding time, and distance computation time in Table IV under $ef = 1000$ and mixed workload. Observe that the time to find base layer entry points is indeed minimal. While distance computation dominates in most datasets (e.g., WIT), in some datasets (e.g., Sift), the overhead of other costs, including graph traversal and queue management, is also significant.

Comparison with Oracle-HNSW. Following [9], we compare FRASP with Oracle-HNSW, which builds a separate HNSW index for each query range–impractical due to its $O(n^3M)$ space cost to support all possible $O(n^2)$ query ranges. We construct Oracle-HNSW for 100 randomly sampled queries

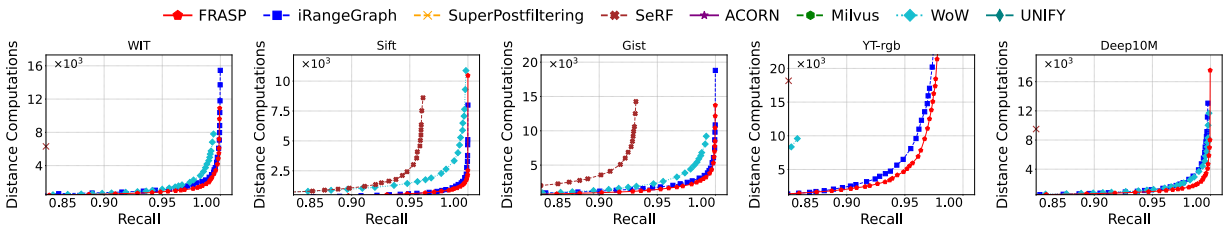


Fig. 7. Distance computations per query vs. recall on the mixed workload (y-axis in thousands; $\times 10^3$ in each panel).

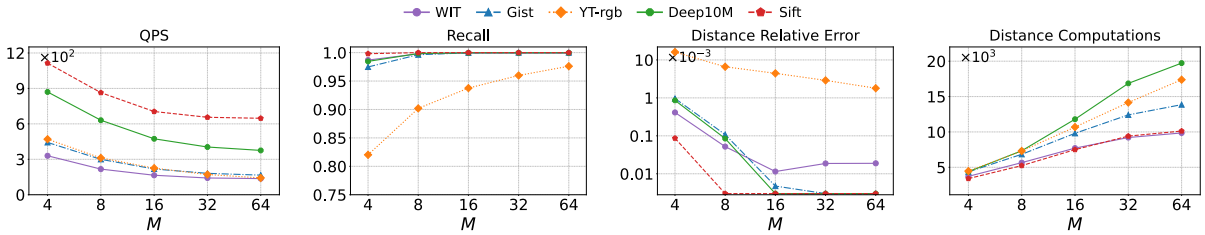


Fig. 8. Vary M on the mixed workload at ef=1000: QPS, recall, distance relative error, and distance computations per query.

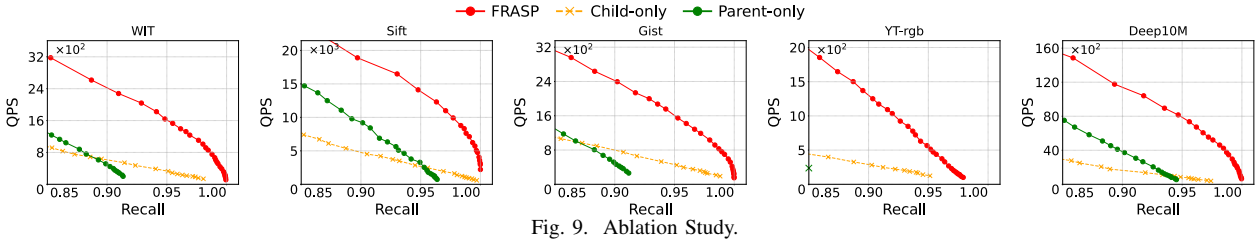


Fig. 9. Ablation Study.

per dataset, as indexing all queries is infeasible. Figure 10 shows QPS-Recall on WIT and Gist (mixed workload): FRASP achieves performance comparable to or even surpassing Oracle-HNSW while using only $O(nM \log n)$ space, demonstrating the efficiency of our approach.

Effect of M. We vary the maximum out-degree $M \in \{4, 8, 16, 32, 64\}$ on the mixed workload at ef=1000 and report QPS, recall, distance relative error, and per-query distance computations in Figure 8. Recall remains high across these settings. QPS, distance relative error and per-query distance computations vary substantially with M. Different datasets show different sensitivities, so the best M can differ by dataset.

Ablation Study. We conduct an ablation study to compare FRASP that uses three segments and their HNSWs, to Child-only (considering only the child segments) and Parent-only (considering only the parent segment). In *Child-only*, the two child HNSWs are disconnected, so a single search entry can traverse only one side; we therefore invoke the same entry-point selection routine *in each child* to obtain a starting vertex per child and begin the layered search from both. As shown in Figure 9 on mixed workload, the performance of Child-only is degraded, indicating that the disconnected HNSWs in the children are not sufficient for high recall. It requires the HNSW of parent segment to introduce inter-edges. The performance of Parent-only also drops because the parent HNSW is built as a suffix or prefix HNSW over a range $[l, r]$ that contains $[L, R]$. The ablation study shows the effectiveness of our design to use three segments and their HNSWs to achieve high QPS and recall over existing methods as reported above.

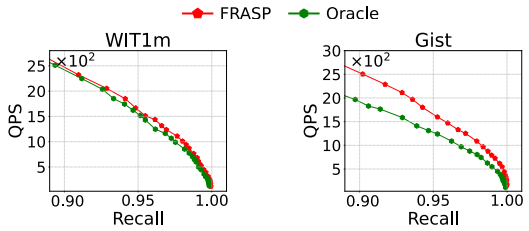


Fig. 10. FRASP vs. Oracle-HNSW under mixed workloads.

VII. Conclusion

In this paper, we propose FRASP which leverages a first-split range strategy combined with suffix and prefix HNSW indexes to enable fast and accurate RFANNS. By consulting only three segment-specific graph indexes, we develop a unified search process that efficiently restricts exploration to in-range objects without redundant computations, reducing the computation cost per object. We also design prefix/suffix max arrays to quickly locate relevant HNSW layers and entry points. Our approach effectively balances the trade-off between query efficiency and result quality, and demonstrates excellent performance across various datasets and query workloads. For future work, we plan to extend our methods to support more complex range-filtering queries, such as queries on dynamic vector data and queries with multiple attributes.

Acknowledgment

This work is supported by grants from the Research Grants Council of Hong Kong Special Administrative Region, China (No. PolyU 15205224) and from ByteDance (P0063921).

REFERENCES

[1] H. Li, T. N. Chan, M. L. Yiu, and N. Mamoulis, "FEXIPRO: fast and exact inner product retrieval in recommender systems," in *SIGMOD Conference*. ACM, 2017, pp. 835–850.

[2] M. Grohe, "word2vec, node2vec, graph2vec, x2vec: Towards a theory of vector embeddings of structured data," in *PODS*. ACM, 2020, pp. 1–16.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *NeurIPS*, 2020.

[4] W. Li, C. Feng, D. Lian, Y. Xie, H. Liu, Y. Ge, and E. Chen, "Learning balanced tree indexes for large-scale vector retrieval," in *KDD*. ACM, 2023, pp. 1353–1362.

[5] H. Deguchi, T. Watanabe, Y. Matsui, M. Utiyama, H. Tanaka, and E. Sumita, "Subset retrieval nearest neighbor machine translation," in *ACL (1)*. Association for Computational Linguistics, 2023, pp. 174–189.

[6] N. Rekabsaz, O. Lesota, M. Schedl, J. Brassey, and C. Eickhoff, "Tripclick: The log files of a large health web search engine," in *SIGIR*. ACM, 2021, pp. 2507–2513.

[7] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[8] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *IEEE Trans. Big Data*, vol. 12, no. 2, pp. 346–361, 2025.

[9] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen, "iRangeGraph: improvising range-dedicated graphs for range-filtering nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 239:1–239:26, 2024.

[10] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng, "SeRF: segment graph for range-filtering approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 69:1–69:26, 2024.

[11] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, and C. Xie, "Milvus: A purpose-built vector data management system," in *SIGMOD Conference*, 2021, pp. 2614–2627.

[12] Q. Zhang, S. Xu, Q. Chen, G. Sui, J. Xie, Z. Cai, Y. Chen, Y. He, Y. Yang, F. Yang, M. Yang, and L. Zhou, "VBASE: unifying online vector similarity search and relational queries via relaxed monotonicity," in *OSDI*. USENIX Association, 2023, pp. 377–395.

[13] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *EMNLP*. ACL, 2014, pp. 1532–1543.

[14] C. Schuhmann, R. Vencu, R. Beaumont, R. Kaczmarczyk, C. Mullis, A. Katta, T. Coombes, J. Jitsev, and A. Komatsuzaki, "LAION-400M: open dataset of clip-filtered 400 million image-text pairs," *CoRR*, vol. abs/2111.02114, 2021. [Online]. Available: https://arxiv.org/abs/2111.02114

[15] Y. Lin, K. Zhang, Z. He, Y. Jing, and X. S. Wang, "Survey of filtered approximate nearest neighbor search over the vector-scalar hybrid data," *CoRR*, vol. abs/2505.06501, 2025.

[16] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, pp. 1–27, 2024.

[17] Z. Wang, J. Zhang, and W. Hu, "WoW: A window-to-window incremental index for range-filtering approximate nearest neighbor search," in *Proc. ACM SIGMOD Int. Conf. Management of Data*. New York, NY, USA: ACM, 2026.

[18] A. Liang, P. Zhang, B. Yao, Z. Chen, Y. Song, and G. Cheng, "UNIFY: Unified index for range filtered approximate nearest neighbors search," *Proc. VLDB Endow.*, vol. 18, no. 4, pp. 1118–1130, Dec. 2024.

[19] M. de Berg, O. Cheong, M. J. van Kreveld, and M. H. Overmars, *Computational geometry: algorithms and applications, 3rd Edition*. Springer, 2008.

[20] Y. Xiang, J. Wang, C. Feng, and J. Shi, "FRASP: Code and Technical Report," https://github.com/wjy-yy/FRASP, 2026.

[21] Y. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, "Approximate nearest neighbor algorithm based on navigable small world graphs," *Information Systems*, vol. 45, pp. 61–68, 2014.

[22] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast approximate nearest neighbor search with the navigating spreading-out graph," *Proc. VLDB Endow.*, vol. 12, no. 5, pp. 461–474, 2019.

[23] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "Diskann: Fast accurate billion-point nearest neighbor search on a single node," *Advances in neural information processing Systems*, vol. 32, 2019.

[24] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 1964–1978, 2021.

[25] A. Beygelzimer, S. Kakade, and J. Langford, "Cover trees for nearest neighbor," in *ICML*, 2006, pp. 97–104.

[26] Y. Gu, Z. Napier, Y. Sun, and L. Wang, "Parallel cover trees and their applications," in *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures*, 2022, pp. 259–272.

[27] Q. Huang, J. Feng, Y. Zhang, Q. Fang, and W. Ng, "Query-aware locality-sensitive hashing for approximate nearest neighbor search," *Proceedings of the VLDB Endowment*, vol. 9, no. 1, pp. 1–12, 2015.

[28] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *STOC*. ACM, 1998, pp. 604–613.

[29] Y. Tao, K. Yi, C. Sheng, and P. Kalnis, "Efficient and accurate nearest neighbor and closest pair search in high-dimensional space," *ACM Transactions on Database Systems (TODS)*, vol. 35, no. 3, pp. 1–46, 2010.

[30] Q. Chen, B. Zhao, H. Wang, M. Li, C. Liu, Z. Li, M. Yang, and J. Wang, "Spann: Highly-efficient billion-scale approximate nearest neighborhood search," *NeurIPS*, vol. 34, pp. 5199–5212, 2021.

[31] C. Fu, C. Wang, and D. Cai, "High dimensional similarity search with satellite system graph: Efficiency, scalability, and unindexed query compatibility," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 8, pp. 4139–4150, 2021.

[32] B. Harwood and T. Drummond, "Fanng: Fast approximate nearest neighbour graphs," in *CVPR*, 2016, pp. 5713–5722.

[33] F. André, A.-M. Kermarrec, and N. Le Scouarnec, "Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan," *Proc. VLDB Endow.*, vol. 9, no. 4, pp. 288–299, 2015.

[34] J. Gao and C. Long, "Rabitq: quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, pp. 1–27, 2024.

[35] R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, and S. Kumar, "Accelerating large-scale inference with anisotropic vector quantization," in *ICML*, vol. 119, 2020, pp. 3887–3896.

[36] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011.

[37] K. Echihabi, T. Palpanas, and K. Zoumpatianos, "New trends in high-d vector similarity search: Ai-driven, progressive, and distributed," *Proc. VLDB Endow.*, vol. 14, no. 12, pp. 3198–3201, 2021.

[38] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, and X. Lin, "Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement," *IEEE Transactions on Knowledge and Data Engineering*, vol. 32, no. 8, pp. 1475–1488, 2019.

[39] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "An efficient and robust framework for approximate nearest neighbor search with attribute constraint," in *NeurIPS*, 2023.

[40] S. Gollapudi, N. Karia, V. Sivashankar, R. Krishnaswamy, N. Begwani, S. Raz, Y. Lin, Y. Zhang, N. Mahapatro, P. Srinivasan, A. Singh, and H. V. Simhadri, "Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters," in *WWW*, 2023, pp. 3406–3416.

[41] J. Mohoney, A. Pacaci, S. R. Chowdhury, A. Mousavi, I. F. Ilyas, U. F. Minhas, J. Pound, and T. Rekatsinas, "High-throughput vector similarity search in knowledge graphs," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–25, 2023.

[42] Y. Matsui, R. Hinami, and S. Satoh, "Reconfigurable inverted index," in *ACM MM*, 2018, pp. 1715–1723.

[43] C. Zuo and D. Deng, "Arkgraph: All-range approximate k-nearest-neighbor graph," *Proceedings of the VLDB Endowment*, vol. 16, no. 10, pp. 2645–2658, 2023.

[44] J. Engels, B. Landrum, S. Yu, L. Dhulipala, and J. Shun, "Approximate nearest neighbor search with window filters," in *International Conference on Machine Learning*. PMLR, 2024, pp. 12 469–12 490.

[45] P. Indyk and H. Xu, "Worst-case performance of popular approximate nearest neighbor search implementations: Guarantees and limitations," in *NeurIPS*, 2023.
[46] A. Rubinstein, "Hardness of approximate nearest neighbor search," in *STOC*. ACM, 2018, pp. 1260–1268.
[47] M. Aumüller, E. Bernhardsson, and A. Faithfull, "Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Information Systems*, vol. 87, p. 101374, 2020.
[48] WIT, "WIT: Wikipedia-based Image Text Dataset," https://github.com/google-research-datasets/wit, 2025.
[49] Youtube-8M, "Youtube-8M: A Large and Diverse Labeled Video Dataset for Video Understanding Research," https://research.google.com/youtube8m/, 2025.
[50] S. Abu-El-Haija, N. Kothari, J. Lee, P. Natsev, G. Toderici, B. Varadarajan, and S. Vijayanarasimhan, "Youtube-8m: A large-scale video classification benchmark," *arXiv preprint arXiv:1609.08675*, 2016.
[51] ANN-benchmarks, "ANN-Benchmarks," http://ann-benchmarks.com/, 2025.
[52] Deep10M, https://research.yandex.com/blog/benchmarks-for-billion-scale-similarity-search, 2016.
[53] M. Patella and P. Ciaccia, "The many facets of approximate similarity search," in *First International Workshop on Similarity Search and Applications (sisap 2008)*. IEEE, 2008, pp. 10–21.